

RAG Document Chatbot: Complete Architecture & Colab Guide

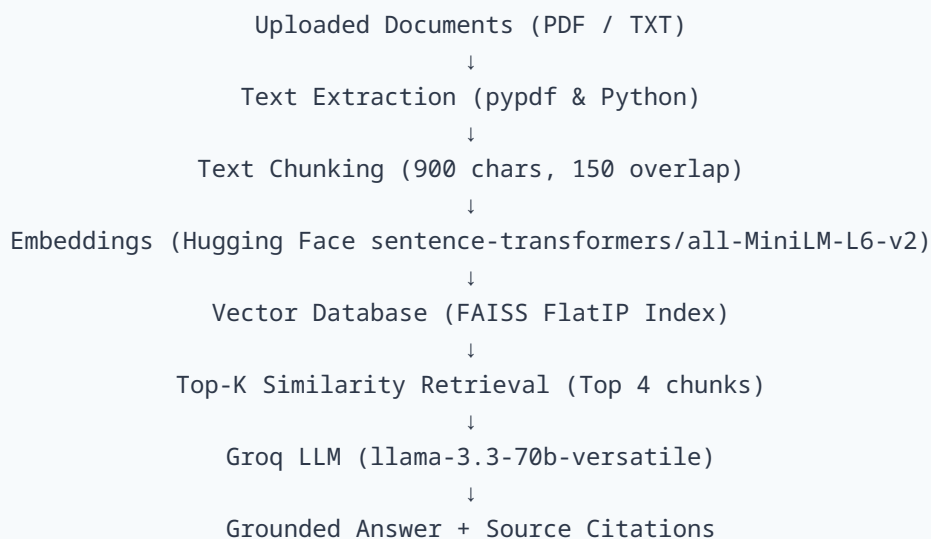
Step-by-step documentation for building a Retrieval-Augmented Generation Chatbot using Python, Groq, Sentence Transformers, and FAISS.

1. Introduction & Core Concepts

Retrieval-Augmented Generation (RAG) is a technique that connects Large Language Models (LLMs) to external document stores without retraining the model. It ensures factual, accurate responses grounded strictly in custom uploaded documents.

- **Embedding:** A vector representation of text capturing semantic meaning.
- **Vector Database (FAISS):** An index designed for ultra-fast similarity searches over high-dimensional vector arrays.
- **Document Chunking:** Dividing long documents into small overlapping passages (e.g., 900 characters) to preserve local context.
- **Prompt Grounding:** Supplying retrieved context into the LLM prompt and forcing the model to rely only on provided facts.

2. RAG Workflow Architecture



3. Vector Database Comparison

Feature	FAISS (faiss-cpu)	ChromaDB
Installation	Extremely lightweight, zero SQLite issues.	Requires heavy dependencies & SQLite3 updates.
Performance	Highly optimized C++ engine for CPU execution.	Good, but higher memory footprint.
Streamlit Cloud Compatibility	100% reliable build and deployment.	Frequent dependency & SQLite build errors.
Selection Reason	FAISS was selected for its speed, low RAM usage, and flawless deployment on free Streamlit Cloud.	

4. Google Colab Implementation (13 Steps)

Cell 1: Install Dependencies

```
!pip install --quiet groq sentence-transformers faiss-cpu pypdf numpy
```

Cell 2: Import Libraries

```
import os, io, numpy as np, faiss
from pypdf import PdfReader
from sentence_transformers import SentenceTransformer
from groq import Groq
```

Cell 3: Load Groq API Key Securely

Security Rule

Always store your key in Colab Secrets under the label `GROQ_API_KEY`. Never write your key directly in code!

```
from google.colab import userdata
try:
    os.environ["GROQ_API_KEY"] = userdata.get('GROQ_API_KEY')
    print("API Key loaded successfully.")
except Exception as e:
    print("Error loading key from Colab Secrets:", e)
```

Cell 4: Define Hyperparameters

```
GROQ_MODEL = "llama-3.3-70b-versatile"
EMBEDDING_MODEL_NAME = "sentence-transformers/all-MiniLM-L6-v2"
CHUNK_SIZE = 900
CHUNK_OVERLAP = 150
TOP_K = 4
```

Cell 5: Load Embedding Model

```
embedder = SentenceTransformer(EMBEDDING_MODEL_NAME)
```

Cell 6: Upload Files

```
from google.colab import files
uploaded = files.upload()
```

Cell 7: Extract Text from Documents

```
extracted_docs = []
for name, content in uploaded.items():
    if name.endswith(".pdf"):
        reader = PdfReader(io.BytesIO(content))
        for page_num, page in enumerate(reader.pages, start=1):
            txt = (page.extract_text() or "").strip()
            if txt:
                extracted_docs.append({"filename": name, "page": page_num, "text": txt})
    elif name.endswith(".txt"):
        txt = content.decode("utf-8", errors="ignore").strip()
        if txt:
            extracted_docs.append({"filename": name, "page": 1, "text": txt})
```

Cell 8: Text Chunking

```
chunks_db = []
for doc in extracted_docs:
    text, filename, page = doc["text"], doc["filename"], doc["page"]
    start, idx = 0, 1
    while start < len(text):
        chunk_text = text[start:start + CHUNK_SIZE]
        chunks_db.append({
            "chunk_id": f"{filename}_p{page}_c{idx}",
            "filename": filename, "page": page, "chunk_num": idx, "text": chunk_text
        })
        idx += 1
        start += (CHUNK_SIZE - CHUNK_OVERLAP)
```

Cell 9: Create FAISS Vector Index

```
corpus_texts = [c["text"] for c in chunks_db]
embeddings = embedder.encode(corpus_texts, convert_to_numpy=True)
faiss.normalize_L2(embeddings)

index = faiss.IndexFlatIP(embeddings.shape[1])
index.add(embeddings)
```

Cell 10: Retrieval Function

```
def retrieve_chunks(query, k=TOP_K):
    q_vec = embedder.encode([query], convert_to_numpy=True)
    faiss.normalize_L2(q_vec)
    distances, indices = index.search(q_vec, k)
    return [chunks_db[idx] for idx in indices[0] if idx != -1 and idx < len(chunks_db)]
```

Cell 11 & 12: Answer Generation & Testing

```
def generate_rag_answer(query, chunks):
    client = Groq(api_key=os.environ.get("GROQ_API_KEY"))
    context = ""
    for idx, c in enumerate(chunks, 1):
        context += f"[{idx}] {c['filename']} (p.{c['page']}): {c['text']}"

    prompt = f"Context:
{context}
User Question: {query}"
    res = client.chat.completions.create(
        model=GROQ_MODEL,
        messages=[
            {"role": "system", "content": "Answer ONLY using context. If unknown, say 'I could not find this information in the uploaded documents.'"},
            {"role": "user", "content": prompt}
        ],
        temperature=0.1
    )
    return res.choices[0].message.content
```